

Mini-Projet Symfony — Gestion des Achats (Procurement System)

🎯 Objectif général

Construire une application **Symfony** permettant à une entreprise de gérer :

- Les **fournisseurs** (suppliers)
- Les **produits**
- Les **demandes d'achat** (Purchase Requests – PR)
- Les **bons de commande** (Purchase Orders – PO)
- La **gestion utilisateurs** (admin + employés)
- Le **cycle de validation** d'une demande (workflow simple)

🎓 Compétences travaillées

- Création d'entités + relations Doctrine
- CRUD complet avec formulaires Symfony
- Templating et composants Twig
- Authentification + Rôles : Admin / Employé
- Messages flash
- Upload de fichiers (devis PDF, pièces jointes)
- Validation de formulaire
- Pagination + recherche (bonus)

Architecture fonctionnelle du projet

1. Utilisateurs

Deux rôles :

→ **ROLE_ADMIN**

- **gérer les fournisseurs**
- **gérer les produits**
- valider les demandes d'achat
- générer les bons de commande

→ **ROLE_USER**

- créer une demande d'achat (PR)
- consulter ses demandes
- ajouter une pièce jointe PDF

2. Modules du projet

📁 **Module Fournisseurs (Suppliers)**

CRUD complet :

Champs	Type
id	int
name	string
email	string
phone	string
address	text

Module Produits (Products)

CRUD + relation avec Supplier :

Champ	Type
id	int
name	string
description	text
price	float
supplier	ManyToOne Supplier

Module Demandes d'Achat (PurchaseRequest – PR)

Champ	Type
id	int
reference	string unique
requester (utilisateur)	ManyToOne User
product	ManyToOne Product
quantity	int
justification	text
attachment (optional)	fichier PDF
status	enum : “draft”, “submitted”, “validated”, “rejected”
created_at	datetime

Bons de commande (PurchaseOrder – PO)

Généré automatiquement quand une PR est validée.

Champ	Type
id	int
order_number	string unique
purchase_request	OneToOne PR
total_amount	calculé = price × quantity
created_at	datetime

ÉNONCÉ DÉTAILLÉ

Étape 1 — Initialisation du projet

1. Créer un nouveau projet Symfony :
2. `symfony new procurement --webapp`
3. Configurer la base de données .env.
4. Installer les dépendances utiles :
5. `composer require annotations orm maker form validator twig asset`

Étape 2 — Création du module Utilisateurs

1. Générer l'entité User
2. `php bin/console make:user`
3. Ajouter :
 - `firstname`
 - `lastname`
 - `role par défaut : ROLE_USER`
4. Mettre en place l'authentification (login / logout)
5. `php bin/console make:auth`
6. Créer une page de profil simple.

Étape 3 — Module Fournisseurs

Créer un CRUD complet :

`php bin/console make:entity Supplier`

`php bin/console make:crud Supplier`

Champs :

- `name` (obligatoire)
- `email` (valide)
- `phone`
- `address`

Contraintes attendues :

- `name` : min 3 caractères
- `email` : `Email()`
- `phone` : Regex optionnel

Étape 4 — Module Produits

Créer un CRUD complet :

`php bin/console make:entity Product`

`php bin/console make:crud Product`

Champs :

- `name`
- `description`
- `price` (`>= 0`)
- `supplier` (ManyToOne)

Contraintes attendues :

- `price` : `Assert\PositiveOrZero`
- relation obligatoire vers un Supplier

Bonus

Styliser le formulaire avec Twig + Bootstrap.

Étape 5 — Module Demandes d'Achat (PR)

Créer l'entité :

php bin/console make:entity PurchaseRequest

Champs :

Champ	Détails
reference	chaîne unique générée automatiquement (ex : PR-2025-0012)
requester	ManyToOne vers User
product	ManyToOne Product
quantity	int >= 1
justification	Text
attachment	Upload fichier PDF dans /uploads/pr/
status	string (draft/submitted/validated/rejected)
created_at	auto

Contraintes :

- PDF uniquement
- quantity >= 1
- justification min 10 caractères

Actions attendues sur les PR

- L'utilisateur (ROLE_USER) peut :
 - créer une PR
 - modifier si status == "draft"
 - soumettre sa PR → change status → "submitted"
- L'admin peut :
 - voir toutes les PR
 - valider / rejeter
 - en cas de validation → créer automatiquement un bon de commande

Étape 6 — Module Bons de Commande (PO)

Créer une entité :

php bin/console make:entity PurchaseOrder

Champs :

Champ	Description
order_number	format "PO-YYYY-XXXX"
purchase_request	OneToOne PR
total_amount	calcul product.price × quantity
created_at	auto

Lors de la validation PR :

- Créer automatiquement un PO
- Recalculer total_amount
- Lier PR ↔ PO

Étape 7 — Templating & UX

Créer les pages suivantes :

Pour tous

- / → tableau de bord simple (nombre PR, PO...)

Pour **ROLE_USER**

- Mes demandes d'achat
- Formulaire de création
- Page de détail PR

Pour **ROLE_ADMIN**

- Liste complète des PR
- Liste des bons de commande
- Boutons “Valider / Rejeter”

Bonus

- Couleurs selon statut :
 - draft → gris
 - submitted → bleu
 - validated → vert
 - rejected → rouge

🔒 Étape 8 — Gestion des rôles & sécurité

✦ Règles attendues :

access_control:

```
- { path: ^/admin, roles: ROLE_ADMIN }  
- { path: ^/pr/new, roles: ROLE_USER }  
- { path: ^/pr/edit, roles: ROLE_USER }  
- { path: ^/pr, roles: ROLE_USER }
```

Template conditionnel :

```
{% if is_granted('ROLE_ADMIN') %}  
  <a href="/admin/pr">Administration</a>  
{% endif %}
```

Étape 9 — Tests fonctionnels (bonus)

Créer quelques tests PHPUnit :

- création PR
- validation PR
- génération PO

Étape 10 — Bonus possibles

- ✓ pagination des listes
- ✓ recherche produit par fournisseur
- ✓ export PDF du PO
- ✓ système de commentaires sur une PR
- ✓ workflow complet (Symfony Workflow)